



MACHINE LEARNING

CCP

Submitted By:

M. Huzaifa Asghar [FA23/BSCS/617]

Zunayra Junaid [FA23/BSCS/632]

Emaan Shaukat [FA23/BSCS/006]

Sadia Sultan [FA23/BSCS/554]

Farheena Aslam [FA23/BSCS/555]

Submitted To:

Miss. Sahar Moin

Section:

A

Chronic Kidney Disease Prediction Using Ensemble Machine Learning Algorithms

ABSTRACT

Chronic Kidney Disease (CKD) is a serious disease affecting millions of people around the world which can progress to kidney failure if not detected early. The challenge of the diagnosis of these conditions is often complicated by the voluminous amount of attributes in a healthcare data, the lack of data and the complicated correlation between the features. The analysis of such data and prediction using machine learning techniques turns out to be an effective solution.

This project is an end-to-end machine learning based healthcare prediction system to detect the Chronic Kidney Disease using the help of ensemble learning algorithms. The attributes of the data used for the model were preprocessed from the UCI Machine Learning Repository, including handling missing values, encoding of categorical data, normalization, and splitting the data into the training and testing sets. Exploratory Data Analysis (EDA) was performed to search for the significant patterns, distribution of features and correlation between medical attributes.

Several ensemble learning methods viz: Random Forest, Gradient boosting, AdaBoost, Voting Classifier and Stacking Classifier were implemented and performance metrics such as Accuracy, Precision, Recall, F1-Score, Confusion Matrix and Cross Validation Accuracy were used to evaluate the methods. All models were compared to each other and the best model was selected for deployment.

A web-based prediction system was built using Flask to enable patients to enter their clinical parameters and be able to get the real-time prediction of their risk of CKD. The end-to-end system demonstrates the possibilities of machine learning in the field of healthcare for early disease detection, resulting in better decisions for patient care.

Table of Contents

1. Introduction
 - i. Background
 - ii. Problem Statement
 - iii. Objectives
2. Literature Review
3. Dataset Description
 - i. Dataset Source
 - ii. Dataset Information
 - iii. Important Features
4. System Methodology
 - i. Workflow Overview
5. Exploratory Data Analysis
 - i. Dataset Overview
 - ii. Missing Values Analysis
 - iii. Class Distribution
 - iv. Numeric Feature Distribution
 - v. Correlation Heatmap
 - vi. Boxplots by Class
 - vii. Categorical Feature Analysis
6. Data Preprocessing
 - i. Handling Missing Values
 - ii. Encoding Categorical Features
 - iii. Target Variable Encoding
 - iv. Feature Scaling and Normalization
 - v. Train-Test Split
 - vi. Saving Preprocessing Objects
7. Model Development
 - i. Random Forest Classifier
 - ii. Gradient Boosting Classifier
 - iii. AdaBoost Classifier
 - iv. Voting Classifier
 - v. Stacking Classifier
8. Model Training
 - i. Cross Validation
9. Model Evaluation
 - i. Accuracy
 - ii. Precision
 - iii. Recall
 - iv. F1-Score
 - v. Confusion Matrix
10. Model Comparison
 - i. Model Comparison Table
 - ii. Model Comparison Graph
 - iii. Best Model Selection
11. Feature Importance Analysis
12. Flask Web Application Development

- i. Technologies Used
 - ii. Application Workflow
 - iii. Input Parameters
 - iv. Prediction Output
 - v. Flask Backend Code
- 13. Real-Time Prediction Example
- 14. Challenges Faced
- 15. Solution Implemented
- 16. Software Engineering Practices
- 17. Future Enhancement
- 18. Conclusion
- 19. References
- 20. Appendices

1. Introduction

1.1. Background

Chronic Kidney Disease (CKD) is one of the primary reasons for significant health problems in the world. Chronic kidney disease (CKD) is when kidneys slowly stop working and cannot remove waste products and extra fluids from the blood. Untreated, the disease could lead to kidney failure, which will need dialysis or a kidney transplant.

The early diagnosis of CKD is of particular importance as it is possible to slow the progression of CKD by choosing appropriate treatment and a healthy lifestyle. But these traditional diagnosis methods are mainly based on subjective interpretation of several clinical parameters that are time consuming and are less accurate.

In the healthcare industry, Machine Learning (ML) techniques can be used to automatically process clinical data and accurately forecast disease risks. Ensemble learning algorithms are especially effective since they generally use multiple models in order to increase the reliability of prediction and decrease overfitting.

To create a complete machine learning model for the prediction of CKD using the ensemble learning techniques and web based deployment framework.

1.2. Problem Statement

Diagnosing CKD based on clinical information can be challenging due to:

- Large number of medical attributes
- Inconsistencies in data from healthcare and missingness
- Interactions between clinical characteristics
- Reliability of diagnosis is low

The standard methods of diagnosis might not be able to capture the hidden patterns in healthcare data. Thus, an intelligent predictive system which can analyze patient information and predict CKD accurately is needed.

1.3. Objectives

This project aims at achieving the following:

- To perform Exploratory Data Analysis (EDA) on the dataset with the aim of analyzing the data in the context of CKD.
- To clean, impute, encode and normalize healthcare data prior to preprocessing it.
- To perform ensemble machine learning algorithms for predicting CKD.
- To assess the performance of a model with several measures.
- To compare ensemble learning algorithms and identify the best model.
- To implement the code for this real time web application for predicting CKD with the Flask web framework.
- To create a reproducible and deployment-ready machine learning pipeline.

2. Literature Review

The application of machine learning in healthcare has been tremendous, notably in disease prediction and diagnosis. There are several studies that have investigated the application of classification algorithms and ensemble learning methods to predict CKD.

Paper 1

The classification accuracy was found to be high with the use of Random Forest and Support vector machine algorithm for prediction of CKD. Random Forest was determined to be better due to its ability to cope with noisy medical data and missing data.

Paper 2

The rest of the study employed ensemble boosting method including AdaBoost and Gradient Boosting. The study reported improving the prediction performance by federating weak learners into strong predictive models.

Paper 3

The inclusion of feature selection and model optimization yields the near perfect classification results of a health care prediction system involving the use of ensemble learning.

Literature Comparison Table

Describe the algorithms used in the research paper, how accurate is it, and what the limitations are.

Research Paper	Algorithm Used	Accuracy	Limitation
Paper 1	Random Forest	98%	Limited dataset size
Paper 2	AdaBoost	97%	Computational complexity
Paper 3	Ensemble Learning	99%	Lack of deployment

Literature Review Summary

The literature shows that ensemble learning methods are superior to each single classifier in terms of accuracy in healthcare prediction systems due to the ability to reduce the overfitting phenomenon, enhance generalization capability and offer strong predictions for complex medical datasets.

3. Dataset Description

3.1. Dataset Source

```
print("Total Records :", df.shape[0])
print("Total Features:", df.shape[1])
```

The data set from UCI Machine Learning Repository was utilized in this project. It includes information about kidney health, such as clinical and lab data.

Dataset Link:

https://archive.ics.uci.edu/ml/datasets/chronic_kidney_disease

3.2. Dataset Information

Property	Value
Total Records	400
Total Features	25
Data Type	Quantitative / Qualitative
Dataset Type	Healthcare

3.3. Important Features

Feature	Description
age	Patient Age
bp	Blood Pressure
sg	Specific Gravity
al	Albumin
su	Sugar
Random blood glucose.Random Blood Glucose (bgr)	
bu	Blood Urea
sc	Serum Creatinine
hemo	Hemoglobin
A blood cell's volume of the total blood volume.	
htn	Hypertension
dm	Diabetes Mellitus
class	CKD Status

4. System Methodology

The proposed system is complete machine learning process, from data collection, preprocessing of data, model training, model testing, deployment, to prediction in real time.

4.1 Workflow Overview

The process of the system is as follows:

1. Dataset Collection
2. Exploratory Data Analysis
3. Data Preprocessing
4. Ensemble Model Training
5. Model Evaluation
6. Best Model Selection

- 7. Model Serialization
- 8. Flask Web Integration
- 9. Real-Time Prediction

5. Exploratory Data Analysis (EDA)

Exploratory Data Analysis was performed to get an understanding of the data, identify missing data, make plots of the distribution of the features, and explore the relationships between medical parameters.

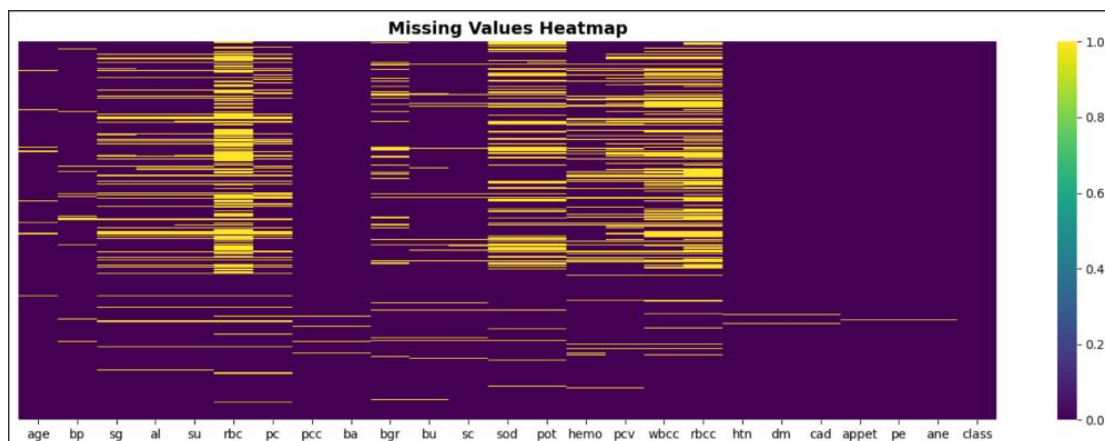
5.1. Dataset Overview

```
print("Total Records :", df.shape[0])  
print("Total Features:", df.shape[1])
```

The data set consists of 400 patient records and several clinical variables which are associated with kidney disease.

5.2. Missing Values Analysis

Missing values of the data were displayed using a heat map.

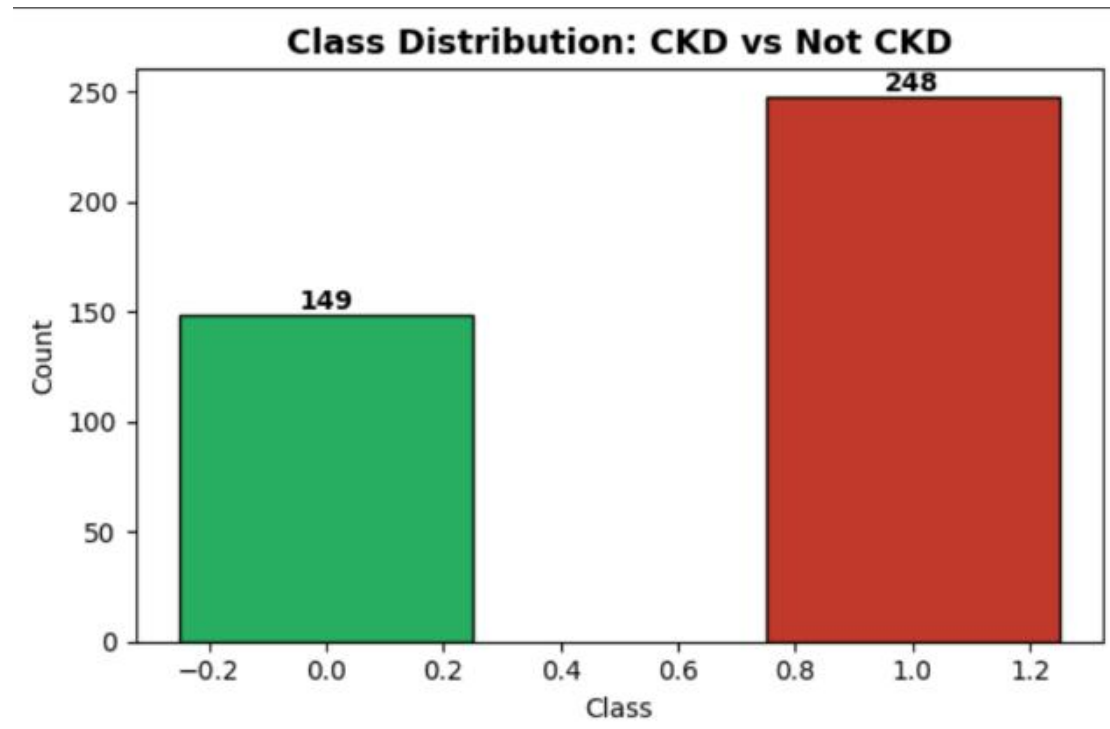


Interpretation

There is a number of missing data in various features. In medical data, there can be missing data as some tests are not always performed on each patient. It is essential to deal with missing data to arrive at the right predictions.

5.3. Class Distribution

A bar graph was used for the analysis of the distribution of CKD and non-CKD class.

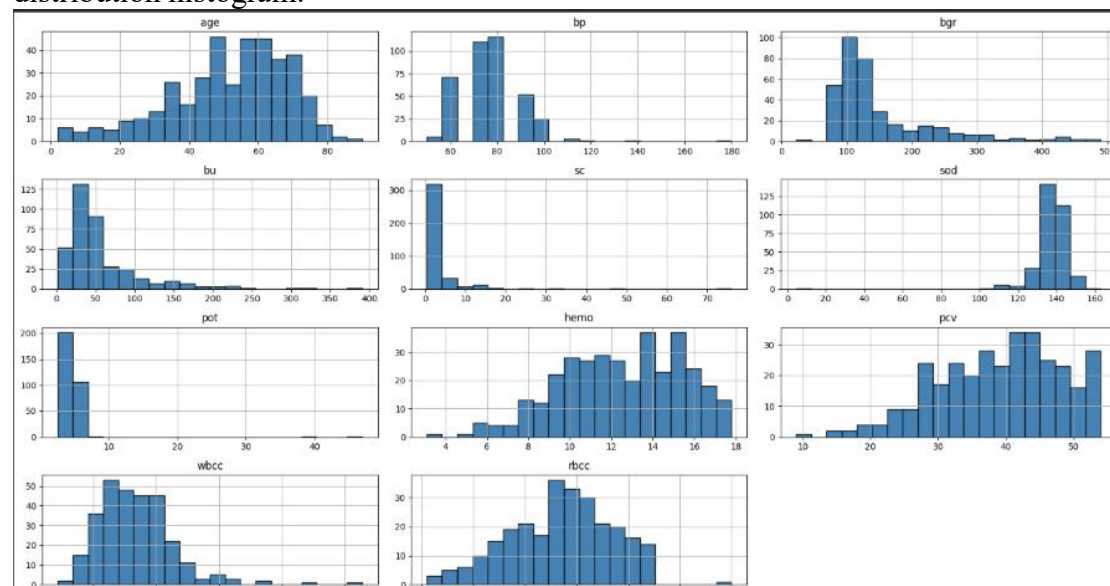


Interpretation

Records from both CKD and non-CKD cases are given. It is important to know the distribution of data among classes because if the data is highly skewed it can affect the performance of the model.

5.4. Numeric Feature Distribution

Histograms were created of significant numeric attributes. To represent numeric attributes of a distribution in a distribution histogram. To display numeric features of a distribution histogram.

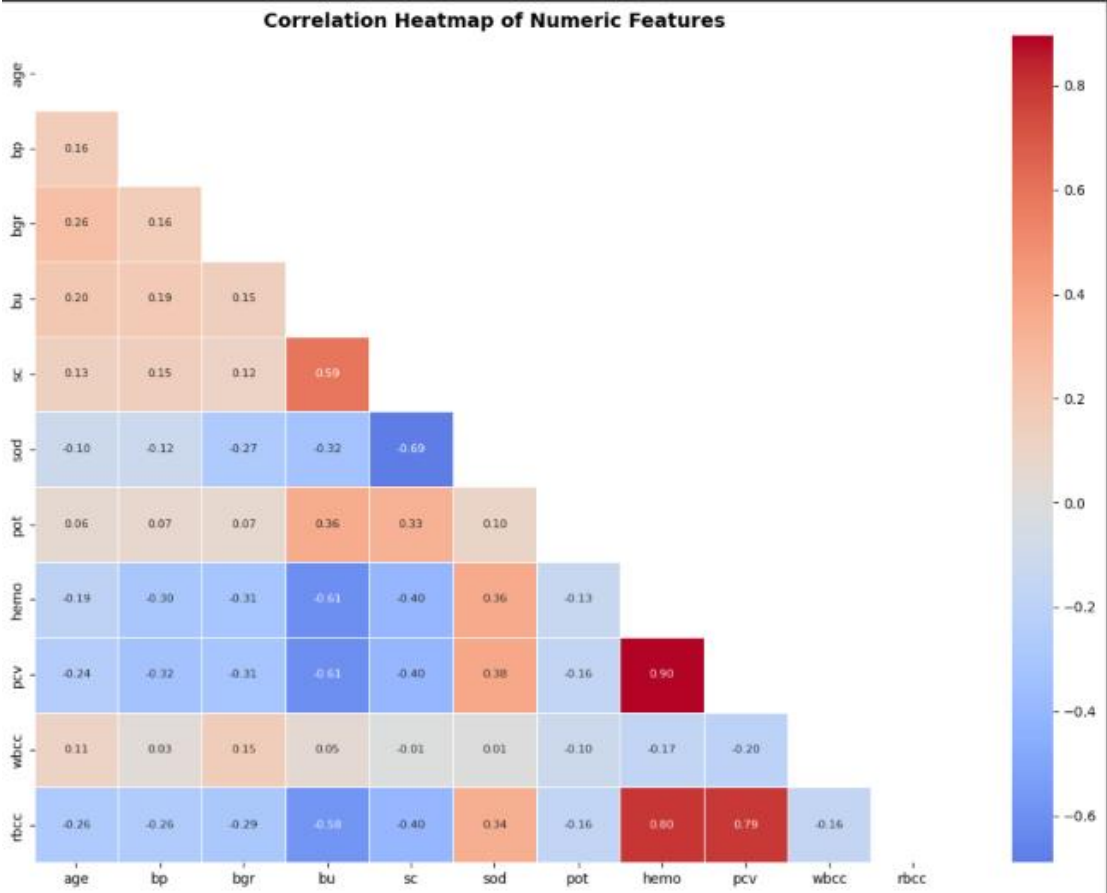


Interpretation

Various medical characteristics showed skewed distribution and abnormal range indicating significant differences between healthy and the CKD patients.

5.5. Correlation Heatmap

A data correlation matrix in numeric format was developed.

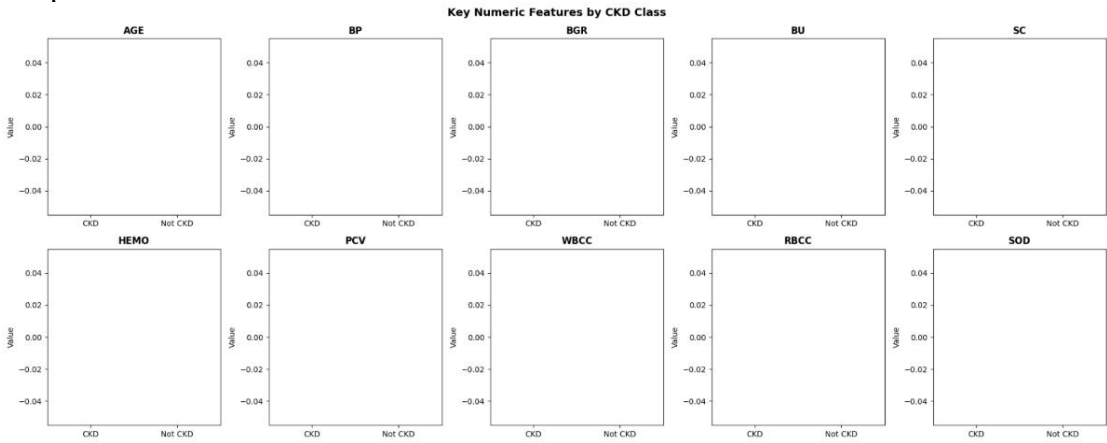


Interpretation

The correlation heatmap illustrates high associations between the serum creatinine, blood urea, hemoglobin and packed cell volume. These properties can be important to predict the development of CKD.

5.6. Boxplots by Class

Distribution for features was compared among the CKD and non-CKD patients using boxplots.

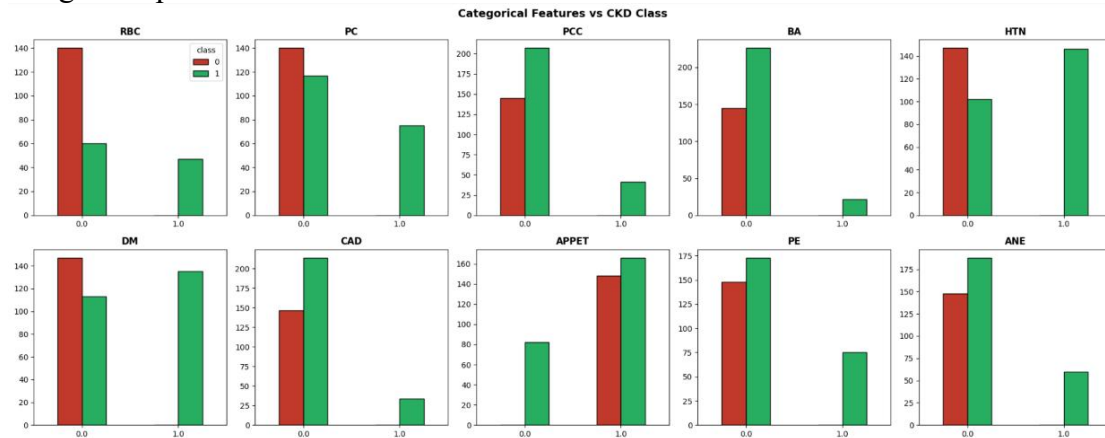


Interpretation

CKD patients typically have abnormal distribution of blood pressure, serum creatinine and hemoglobin.

5.7. Categorical Feature Analysis

Features such as hypertension, diabetes, anemia and appetite were plotted using categorical plots.



Interpretation

There was a significant association between Chronic Kidney Disease and hypertension or diabetes in patients. Patients with hypertension and/or diabetes had significantly higher rates of Chronic Kidney Disease than did normal patients. Similarly, the prevalence of abnormal RBCs, loss of appetite and anemia/pedal edema was more in the CKD patients. The contribution of medical conditions and of symptoms related to lifestyle are strong in the categorical relationships to predict CKD.

The categorical analysis also showed that binary clinical indicators are very effective for classification by machine learning as this effectively separates the CKD and non-CKD groups.

6. Data Preprocessing

Preprocessing of data is one of the crucial steps in the machine learning process, as medical data tends to be incomplete, inconsistent, categorical, and noise-filled. The proper preprocessing improves the model accuracy, training capability of the model and prediction reliability.

The CKD dataset had to undergo some preprocessing steps prior to using it for machine learning training.

6.1. Handling Missing Values

The data set had missing data in the following attributes:

- Red Blood Cells
- Leucemia maligna (acute or chronic)
- Packed Cell Volume
- Serum Creatinine

- Sodium
- Potassium

Lack of data can have a negative impact on the performance of the model and could yield incorrect predictions.

Technique Used

SimpleImputer(strategy='median') was used to impute the median.

Reason for Choosing Median:

The mean is more sensitive to the presence of outliers than is median. Medical datasets may contain values that are statistically abnormal, and median imputation will not heavily rely on these for the imputed values.

Code Snippet:

```
imputer = SimpleImputer(strategy='median')
X_imputed = pd.DataFrame(imputer.fit_transform(X), columns=X.columns)
```

6.2. Encoding Categorical Features

Machine learning algorithms cannot be directly applied with categorical values that are available in the text. Therefore, the categorical features were converted to numeric ones.

Original Value	Encoded Value
normal	0
abnormal	1
present	1
notpresent	0
yes	1
no	0
good	1
poor	0

Code Snippet

```
binary_map = {
    'normal': 0,
    'abnormal': 1,
    'present': 1,
    'notpresent': 0,
    'yes': 1,
    'no': 0,
    'good': 1,
    'poor': 0
}
```

6.3. Target Variable Encoding

The target class was converted to two values: "1" and "0".

Class	Encoded Value
ckd	1
notckd	0

Code

```
df['class'] = df['class'].map({'ckd': 1, 'notckd': 0})
```

6.4. Feature Scaling and Normalization

Numerical features were of different scales. For example:

- Blood Glucose values were very large
- The values of decimal numbers were called Specific Gravity.

Algorithms can be skewed towards larger numbers, unless they are normalized.

Technique Used

Standardization using StandardScaler.

Code Snippet

```
scaler = StandardScaler()
X_scaled = pd.DataFrame(
    scaler.fit_transform(X_imputed),
    columns=X_imputed.columns
)
```

6.5. Train-Test Split

The data set was divided into training and test sets.

Dataset Portion	Percentage
Training Set	80%
Testing Set	20%

Reason

- Data for training models are used.
- Data used in testing are data that is not seen by the model.

Code

Uses train_test_split to split the data into training and test sets. Splits the data into training and test sets with train_test_split.

```
X_scaled,
y,
test_size=0.2,
random_state=42,
stratify=y)
```

6.6. Saving Preprocessing Objects

Pickle serialization was used to save the objects in the Pickle protocol.

Saved Objects

- scaler.pkl
- selected_features.pkl

Benefits

- Reusable preprocessing
- Consistent deployment pipeline
- Faster real-time prediction

7. Model Development

The prediction of CKD was done using ensemble machine learning algorithms. The techniques that combine multiple weak learners to achieve superior predictive performance, stability and robustness are referred to as ensemble methods.

The following models were created:

- Random Forest
- Gradient Boosting
- AdaBoost
- Voting Classifier
- Stacking Classifier

7.1. Random Forest Classifier

One of the ensemble learning algorithms, which is a combination of multiple decision trees is Random Forest.

Working Principle

- Subsets of multiple trees are trained on random subsets.
- Each tree makes a prediction
- Majority voting) is used to make the final prediction.

Advantages

- High accuracy
- Reduces overfitting
- Is able to manage when there is missing information
- Provides feature importance

Code

```
rf_model = RandomForestClassifier(  
    n_estimators=100,  
    random_state=42,  
    n_jobs=-1  
)
```

7.2. Gradient Boosting Classifier

Gradient Boosting creates trees one after the other, with the aim of correcting the mistakes of the trees before.

- High predictive performance
- Handles complex patterns
- Reduces bias

Code

```
gb_model = GradientBoostingClassifier(  
    n_estimators=100,  
    learning_rate=0.1,  
    random_state=42  
)
```

7.3. AdaBoost Classifier

AdaBoost focuses particularly on other samples which were misclassified during training.

Advantages

- Improves weak learners
- Good generalization
- A very good identification source.

Code

```
ada_model = AdaBoostClassifier(  
    n_estimators=100,  
    random_state=42  
)
```

7.4. Voting Classifier

Voting Classifier is a combination of predictions from several models.

Working Principle

They are independent of each other.

The final output is based on the majority vote.

Base Models Used

Random Forest

Gradient Boosting

AdaBoost

Code

```
VotingClassifier(  
    estimators=[  
        ('rf', rf_model),  
        ('gb', gb_model),  
        ('ada', ada_model)  
    ],  
    voting='hard'  
)
```

7.5. Stacking Classifier

Stacking is the stacking of several models, whose outputs are used by another model.

Architecture

Base Models

- Random Forest
- Gradient Boosting
- AdaBoost

Meta Learner

- Logistic Regression

Advantages

- Improves prediction accuracy
- Develops understanding of the benefits of different models
- Produces robust predictions

8. Model Training

The preprocessed training set was used for training all models.

Training Pipeline

- Load dataset
- Preprocess data
- Train models
- Save trained models
- Automated Training Script

The train_models.py program fully automates the entire training process.

Benefits

- Reproducibility
- Faster retraining

- Organized workflow

8.1. Cross Validation

5-fold Cross Validation was used to assess the stability of the models.

Formula

The proportion of the correctly classified samples is represented by the accuracy. The accuracy is the percentage of the number of samples correctly classified. $\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$

The cross validation is used in order to ensure the generalizability of models to new data.

Advantages

- Reduces overfitting
- Reliable evaluation
- Stable performance estimation

9. Model Evaluation

The accuracy of the trained models was evaluated by various measures of accuracy.

Evaluation Metrics

- Accuracy
- Precision
- Recall
- F1-Score
- Confusion Matrix

9.1. Accuracy

The accuracy is measured by the number of samples which is correctly predicted.

Formula:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

Where:

TP = True Positive

TN = True Negative

FP = False Positive

FN = False Negative

9.2. Precision

Precision: Number of true CKD cases predicted / number of cases predicted.

Formula:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

9.3. Recall

The number of patients diagnosed as CKD is the number of patients that can be contacted.

Formula:

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

A high recall value is essential in the health care system: If a patient with CKD is unaware of his diagnosis, the treatment will be postponed and his health risk is increased.

9.4. F1-Score

F1-Score is a metric that takes into account Precision and Recall.

Formula:

F1-Score is a metric of the accuracy of the model and its recall. The F1-Score is a metric of precision and recall of the model. The F1 Score is the weighted average of the precision and recall. F1 Score is the harmonic mean of precision and recall.

Importance

F1-Score is useful if the data have class imbalance as it balances both false positives and false negatives.

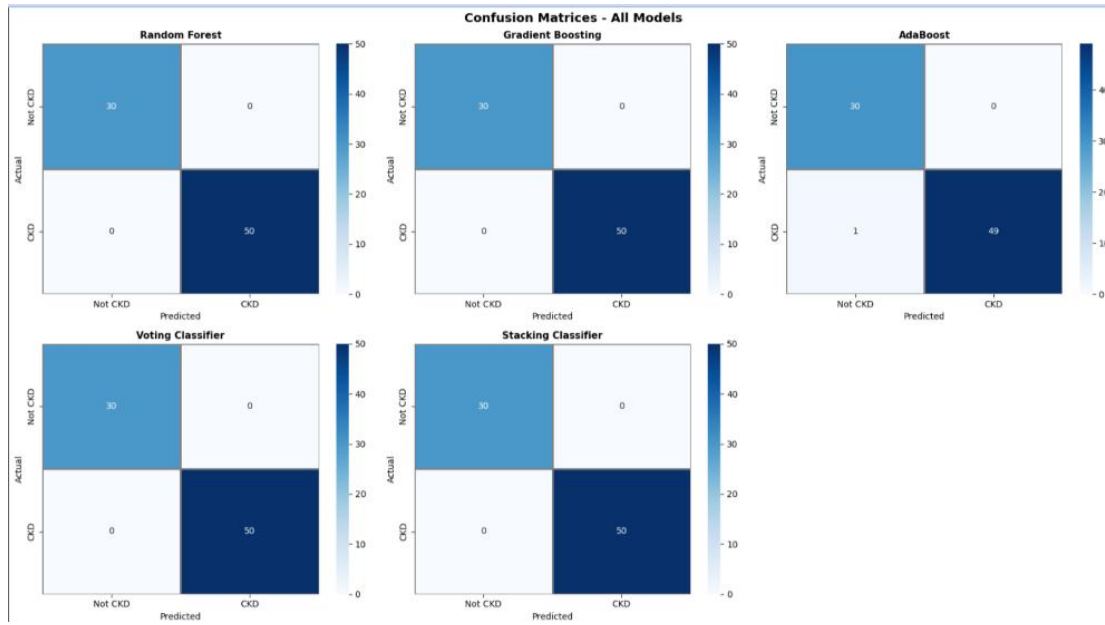
9.5. Confusion Matrix

The results of classification are shown in Confusion Matrix.

AP ratio (actual / predicted) < 1.25 CKD Not CKD

Overall accuracy: 93.75%. The overall accuracy was 93.75% on the last day.

No CKD Subnormal CKD



Interpretation

The other models are shown as accurate with the lowest percentage of misclassified patients in the ensemble models, most of the patients were correctly classified in both the CKD and non-CKD group.

Random Forest and Stacking Classifier had the highest number of correct predictions that indicated good classification's ability.

10. Model Comparison

All the ensemble models were compared with each other using the evaluation metrics.

Compared Models

- Random Forest
- Gradient Boosting
- AdaBoost
- Voting Classifier
- Stacking Classifier

10.1. Model Comparison Table

Model Performance Comparison (sorted by F1-Score):				
	Accuracy	Precision	Recall	F1-Score
Random Forest	1.0000	1.0	1.00	1.0000
Gradient Boosting	1.0000	1.0	1.00	1.0000
Voting Classifier	1.0000	1.0	1.00	1.0000
Stacking Classifier	1.0000	1.0	1.00	1.0000
AdaBoost	0.9875	1.0	0.98	0.9899

10.2. Model Comparison Graph

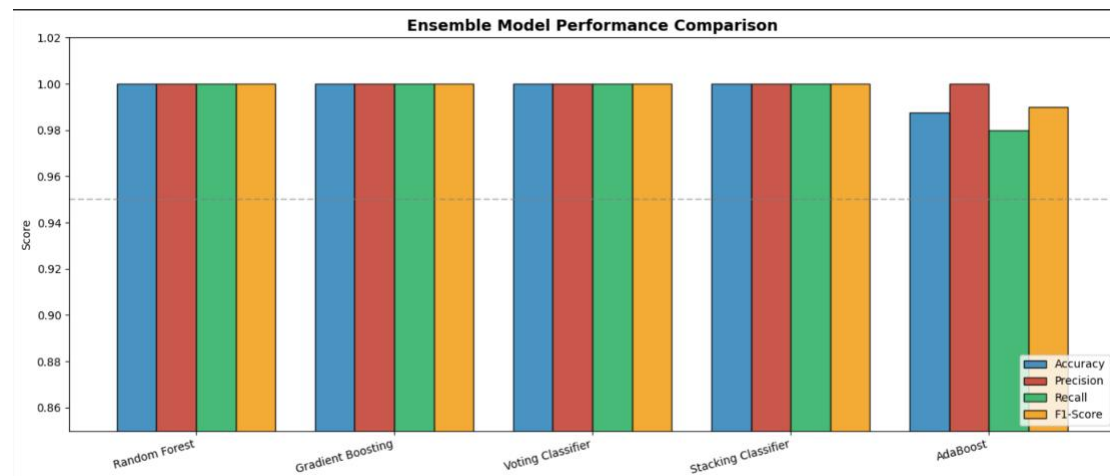
Interpretation

The results of different ensemble learning models used in the CKD data set were compared using the comparison graph, and it is concluded that all the models are effective.

However:

- Random Forest had the best balance of performance and was the outlier.
- Performance of Stacking Classifier was good in terms of generalization.
- The performance of AdaBoost was slightly poorer than others.
- The outcome revealed that the ensemble learning techniques are very effective in predicting the healthcare system.

10.3. Best Model Selection



The best model was chosen according to the following:

- Highest F1-Score
- Stable cross-validation accuracy
- Low misclassification rate
- Better generalization

Selected Model

- Random Forest Classifier

Reasons for Selection

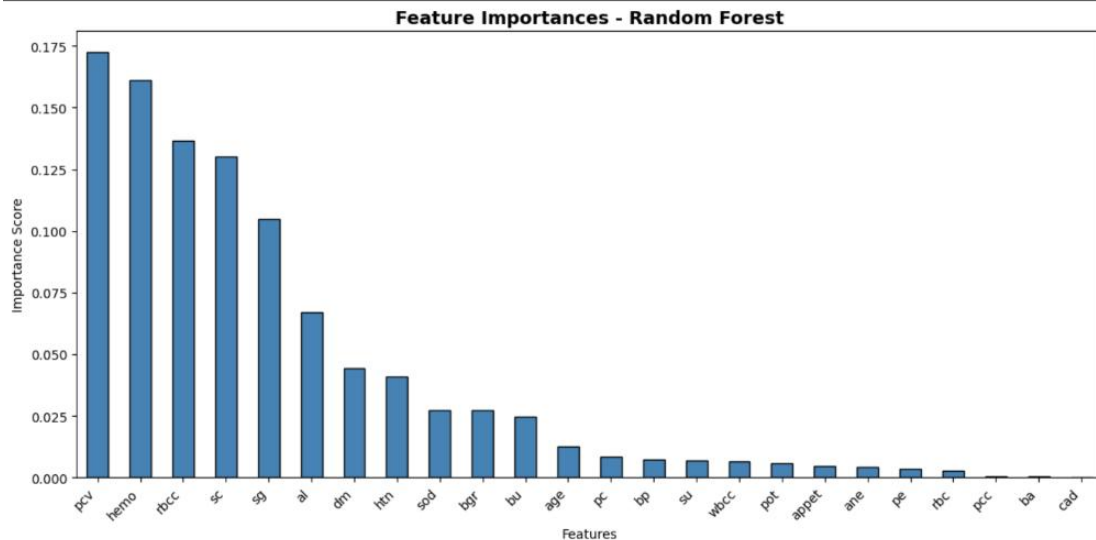
- Excellent prediction accuracy ‘

The various data representations for missing and noisy data were excellent.

- Fast prediction time
- Accurate Feature importance analysis
- Better deployment suitability

11. Feature Importance Analysis

The feature importance analysis is conducted to find the medical attributes which are most important in predicting the presence of CKD.



12. Flask Web Application Development

The development of Web Applications using Flask. Creating Web Applications using Flask.

Flask has been used to build the web based prediction system.

An application for the web is created to collect clinical information for the patient and provide instant prediction of CKD.

12.1. Technologies Used

Technology	Purpose
Flask	Backend Framework
HTML/CSS	Frontend Design
Python	Machine Learning Logic
Pickle	Model Serialization

12.2. Application Workflow

Step 1

User added patient information.

Step 2

The data is passed to the Flask backend in the form.

Step 3

Preprocessing.py is used to preprocess the input data.

Step 4

The trained Random Forest model is used for predicting the risk of CKD.

Step 5

The user is shown the results of the prediction.

12.3. Input Parameters

The following patient parameters are accepted by the Web interface:

- Age
- Blood Pressure
- Specific Gravity
- Albumin
- Sugar
- Red Blood Cells
- Blood Glucose Random
- Blood Urea
- Serum Creatinine
- Hemoglobin
- Hypertension
- Diabetes Mellitus
- Appetite
- Anemia
- Pedal Edema

12.4. Prediction Output

The application displays:

- The results of the test are reported as
- CKD Detected / No CKD Detected.
- Confidence Score
- Risk Level

12.5. Flask Backend Code

Code Snippet

```
@app.route('/predict', methods=['POST'])
def predict():
    form_data = request.form.to_dict()

    input_scaled = preprocess_input(form_data)
    prediction = model.predict(input_scaled)[0]

    return render_template('result.html')
```

13. Real-Time Prediction Example

Parameter	Value
Age	48
Blood Pressure	80
Blood Glucose	121
Hemoglobin	15.4
Prediction Result	

CKD Detected	
Confidence Score	
97.8%	

14. Challenges Faced

There were some problems to be faced in the development of the project.

Major Challenges

Missing Data

There were numerous medical features that lacked values.

There are some problems with ARFF files. Some problems with ARFF files.

Data set was manually parsed because default loaders were not able to format the data.

Feature Scaling

The range of the numerical measurements were very different for the various medical measurements.

Model Selection

A major challenge in the selection of the most reliable ensemble model was carrying out a sufficient evaluation.

15. Solutions Implemented

Challenge	Solution
Median Imputation: This technique is used when data are missing.	
The data comes in various forms: The data is presented in different formats:	
Different Scales	StandardScaler
Model Optimization & Ensemble Learning	

16. Software Engineering Practices

Modular Software Engineering was followed in the project.

- Modules Created
- Module Purpose
- Data Analysis is a topic given in EDA Notebook.
- SQRREW Data in Prereading Notebooks.
- Training Script: Model Training
- Analysis of performance is included in the Evaluation Notebook.
- Flask Application Deployment

17. Future Enhancements

The project could be further improved in the future.

- Possible Enhancements
- Deep Learning Integration

- Cloud Deployment
- Mobile Application
- Combining hospital data with a real-time system. Real time integration of hospital databases.

18. Conclusion

The machine learning based system to predict Chronic Kidney Disease using ensemble learning algorithms was successfully developed in this project.

Data cleaning, missing value imputation, encoding and normalization techniques were used to preprocess the healthcare dataset. Various ensemble methods like Random Forest, Gradient Boosting, AdaBoost, Voting Classifier and Stacking Classifier were created and evaluated.

In all models, Random Forest has the maximum accuracy and has been chosen to be deployed in the Flask app.

The final system provides a user-friendly web interface to instantly predict CKD, and demonstrates the potential for utilizing machine learning in the area of early detection of disease and healthcare decision making.

19. References

Dataset

The Chronic Kidney Disease Dataset can be found in the UCI Machine Learning Repository. Chronic Kidney Disease Dataset can be found in UCI Machine Learning Repository.

Research Paper

This is the Prediction research paper. This is the Prediction research paper.

<https://www.sciencedirect.com/science/article/abs/pii/S1746809423008017>

Documentation

- Scikit-learn Documentation
- Flask Documentation
- Pandas Documentation
- NumPy Documentation

20. Appendix

Appendix A – Project Folder Structure

CKD_Project/

```

|
|— dataset/
|— models/
|— preprocessing/
|— templates/
|— reports/
|
|— 01_EDA.ipynb
|— 02_Preprocessing.ipynb

```

- 03_Model_Training.ipynb
- 04_Model_Evaluation.ipynb
- preprocessing.py
- train_models.py
- app.py
- requirements.txt

The following Appendixes contain a list of the libraries used.

Library	Purpose
Pandas	Data Handling
NumPy	Numerical Computation
Matplotlib	Visualization
Seaborn	Statistical Graphs
Scikit-learn	Machine Learning
Flask	Web Deployment
Pickle	Model Saving
